

Experiment 6: Pattern Classification Using Perceptron Model

Aim

To implement the Perceptron Learning Algorithm for binary pattern classification using Python.

Software / Apparatus Required

- Python 3.x
- VS Code with Jupyter Notebook
- NumPy
- Matplotlib

Theory

The Perceptron is a single-layer neural network used for binary classification. It was introduced by Frank Rosenblatt.

The Perceptron calculates the net input using:

$$\mathbf{z} = \mathbf{W}^T \mathbf{X} + \mathbf{b}$$

where:

- $\mathbf{W}$ = weight vector
- $\mathbf{X}$ = input vector
- $\mathbf{b}$ = bias
- $\mathbf{z}$ = net input

A step activation function is then applied:

$$y = +1, \text{ if } z \geq 0$$

$$y = -1, \text{ if } z < 0$$

The predicted output is compared with the target output. If the prediction is incorrect, the weights and bias are updated using the Perceptron learning rule:

$$\mathbf{W}(\text{new}) = \mathbf{W}(\text{old}) + \eta(\mathbf{T} - \mathbf{Y})\mathbf{X}$$

where:

- η = learning rate
- T = target output
- Y = predicted output

The training process is repeated until all patterns are correctly classified or the maximum number of epochs is reached.

The Perceptron can successfully classify data when the classes are linearly separable.

Algorithm

1. Start.
2. Import NumPy and Matplotlib.
3. Define the input patterns for Class 1 and Class 2.
4. Assign target values of +1 to Class 1 and -1 to Class 2.
5. Initialize the weights and bias.
6. Set the learning rate and maximum number of epochs.
7. Calculate the net input for each training pattern.
8. Apply the step activation function to obtain the predicted output.
9. Compare the predicted output with the target output.
10. If the prediction is incorrect, update the weights and bias.
11. Repeat the process for all patterns until all patterns are correctly classified or the maximum number of epochs is reached.
12. Test the trained Perceptron on the input patterns.
13. Calculate the classification accuracy.
14. Plot the input patterns and decision boundary.
15. Stop.

Applications

1. Email spam detection
2. Optical character recognition (OCR)
3. Medical diagnosis
4. Face recognition
5. Credit approval

These are also listed as applications in the provided experiment material.

Advantages

1. Simple and easy to implement.
2. Fast training for simple classification problems.
3. Efficient for linearly separable datasets.
4. Requires relatively simple computations.

Limitations

1. It cannot solve non-linearly separable problems such as XOR.
2. The basic Perceptron supports binary classification.
3. It requires the training data to be linearly separable for guaranteed convergence.

Conclusion

The Perceptron Learning Algorithm was successfully implemented in Python for binary pattern classification. The model learned a suitable decision boundary between the two linearly separable classes and correctly classified all the given patterns, achieving **100% classification accuracy**.